

PROBLEMA 1 (3 puntos)

Una empresa de desarrollo de videojuegos ha iniciado un nuevo proyecto para crear la aplicación software “Colonos de la ETSISI”, una versión digital de un juego de mesa. En este juego hay un tablero, caracterizado por un nombre y un identificador, el cual está compuesto por casillas hexagonales. De cada una de estas casillas se debe conocer su identificador y las casillas adyacentes a ella. Asuma que cada casilla debe estar adyacente al menos a otra, pudiendo estar una casilla completamente rodeada por otras. De cada casilla también se debe conocer el tipo de recurso (Piedra, Ganado, Trigo o Madera) que tiene asociado, así como su nivel de productividad.

Los jugadores pueden ocupar casillas construyendo en ellas dos tipos de edificios diferentes: ciudades y poblados. No es posible ocupar una casilla sin construir un edificio ni construir un edificio sin ocupar una casilla. Por tanto, a raíz de una ocupación siempre se construye un edificio. En ningún caso podrá haber más de un edificio en una misma casilla (es decir, una casilla no podrá ser ocupada por más de un jugador), aunque los jugadores podrán construir múltiples edificios (cada uno en una casilla diferente). De todo edificio se debe conocer su identificador, su nombre, qué jugador lo ha construido y la casilla que ocupa. Los jugadores podrán utilizar durante su turno uno de sus edificios para recolectar unidades del tipo de recurso asociado a la casilla donde éste haya sido construido. La diferencia entre los dos tipos de edificios es que la forma de recolectar recursos de la ciudad es diferente y más eficiente que la del poblado. Además, los poblados, a diferencia de las ciudades, pueden ser ampliados. Siempre que un jugador amplíe un poblado recolectará recursos con dicho poblado.

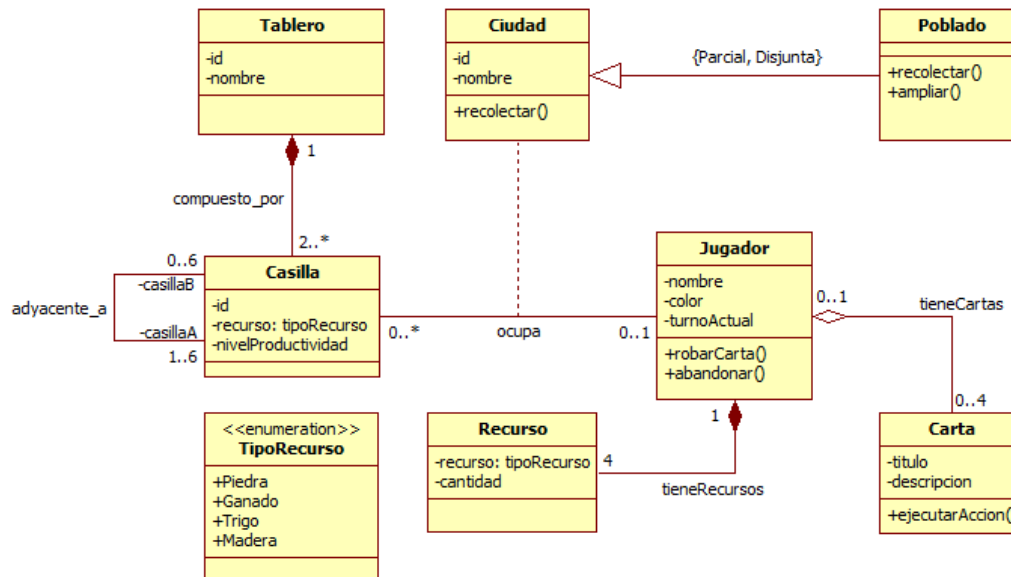
Todo jugador deberá tener un nombre y un color. Además, se deberá conocer en todo momento a qué jugador corresponde el turno actual, así como cuantas unidades de cada tipo de recurso tiene cada jugador. Los jugadores podrán, en determinados momentos, robar cartas hasta tener un máximo de 4. Cada carta tiene un título y un texto que describe la acción que permite ejecutar a los jugadores. La utilización de ciertas cartas requiere a los jugadores robar nuevas cartas. Tenga en cuenta también que, cuando un jugador abandona la partida, sus cartas vuelven a estar disponibles para el resto de jugadores.

Apartado 1.1 (2 puntos)

Elabore el **diagrama de clases de análisis** para modelar la aplicación software descrita teniendo en cuenta ÚNICAMENTE la información aportada por el enunciado, **explicando y justificando claramente las decisiones tomadas** durante su elaboración.

Solución

Se asume que todas las clases tienen un método constructor, un método destructor y un método get y otro set para cada uno de sus atributos

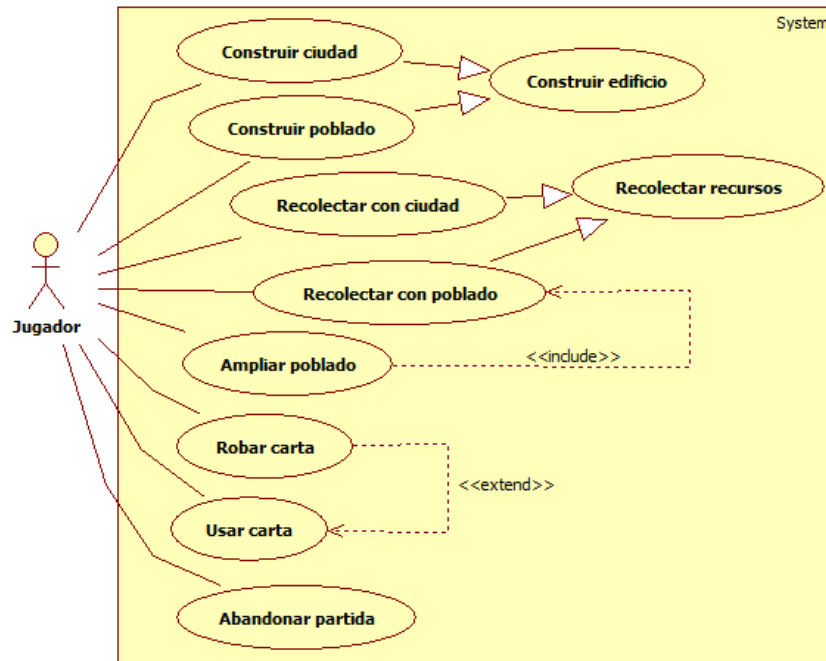


Explicación y justificación de las decisiones tomadas para la elaboración del diagrama de clases de análisis:

- Se ha incluido una clase "Casilla" y una clase "Jugador" conectadas mediante una relación de asociación, fruto de la cual surge la clase de asociación "Ciudad". Esto se ha modelado de esta manera porque siempre que un jugador ocupa una casilla lo hace mediante un edificio. La cardinalidad de la asociación es 0..* en el extremo de "Casilla", porque un jugador puede ocupar múltiples casillas, mientras que en el extremo de "Jugador" es 0..1 porque una casilla o está libre o está ocupada por un solo jugador.
- Para modelar los edificios es suficiente con incluir las clases "Ciudad" y "Poblado", porque no hay ningún atributo, método o relación que tenga "Ciudad" y no tenga "Poblado". Por lo tanto, resulta innecesario crear una tercera clase "Edificio". La herencia entre "Ciudad" y "Poblado" debe ser parcial para poder crear objetos de la clase "Ciudad" y disjunta porque solo hay una clase hija.
- La clase "Poblado" debe tener un método ampliar porque solo los poblados (y no las ciudades) pueden ser ampliadas. La clase "Poblado" también debe redefinir el método recolectar definido en "Ciudad" porque el enunciado dice que poblados y ciudades recolectan de forma diferente.
- La relación entre "Jugador" y "Carta" debe ser una agregación referencial porque si el jugador abandona la partida sus cartas vuelven a estar disponibles.
- La relación entre "Jugador" y "Recurso" se ha modelado con una agregación inclusiva (composición) porque si se elimina un objeto de la clase 'Jugador' no tiene ninguna utilidad para la aplicación almacenar las unidades de recurso que tenía. Las cardinalidades son 1 (extremo "Jugador") y 4 (extremo "Recurso") porque los jugadores pueden tener unidades de cuatro tipos diferentes de recursos y cada conjunto de unidades de un tipo de recurso representado por la clase "Recurso" pertenece a un único jugador.
- La relación entre "Tablero" y "Casilla" se ha modelado con una composición porque un tablero está compuesto por casillas (al menos dos) y una casilla no tiene sentido sin su tablero.

Apartado 1.2 (1 punto)

Elabore el **diagrama de casos de uso** de la aplicación software descrita teniendo en cuenta ÚNICAMENTE los requisitos mencionados en el enunciado, **explicando y justificando claramente las decisiones tomadas durante su elaboración.**



Explicación y justificación de las decisiones tomadas para la elaboración del diagrama de casos de uso:

- Se ha incluido un único actor “Jugador”, ya que el enunciado no indica que ningún otro tipo de usuario o entidad externa interactúe con la aplicación.
- Se han incluido como casos de uso todos los requisitos funcionales descritos en el enunciado (y únicamente estos).
- Se ha establecido una relación de generalización entre el caso de uso “Construir edificio” (caso de uso padre) y “Construir ciudad” y “Construir poblado” (casos de uso hijo), ya que estos casos de uso hijo añaden comportamiento específico al caso de uso padre.
- Se ha supuesto que la recolección de recursos efectuada mediante una ciudad y un poblado, aunque son casos de uso diferentes, tienen un comportamiento parcialmente común, ya que en ambos casos se recolectan unidades del tipo de recurso asociado a la casilla donde se ha construido el edificio. Por este motivo existe un caso de uso padre “Recolectar recursos” y una relación de generalización entre este caso de uso y los casos de uso “Recolectar con ciudad” y “Recolectar con poblado”.
- Se ha utilizado una relación de inclusión (<<include>>) para relacionar ‘Ampliar poblado’ (caso de uso base) con “Recolectar con poblado” (caso de uso incluido), ya que siempre que se ejecuta el primero se ejecuta el segundo.
- Se ha utilizado una relación de extensión (<<extend>>) para relacionar “Usar carta” (caso de uso base) con “Robar carta” (caso de uso que extiende), ya que la ejecución del primero no siempre provoca la ejecución del segundo, sino que esta ejecución es opcional.